Lab Assignment 2

Web Application Security

Authors: Tomás Sobral Teixeira

Eva Sophia Schreyer

1 Report

1.1 Task 1: Regular Expression Denial of Service (ReDoS)

A ReDoS attack is a Denial of Service attack that exploits the fact that most Regex implementations may reach extreme situations that cause them to work very slowly, exponentially related to the input size. The Regex algorithm builds an NFA and then starts making transitions until the end of the input.

In our case, we used the pattern `^(a+)+$` as the password and a few a's followed by another character (e.g. aaaaaaaaaaaaaaaaaaa!) as the user name when registering a client. This causes a ReDoS attack because of a Regex engine feature called backtracking. This feature makes it so that if the input fails to match, the engine goes back to previous positions where it could take a different path. The engine tries this many times until it explores all possible paths. In our expression, we have a long running loop because there were too many paths to explore (as seen bellow).

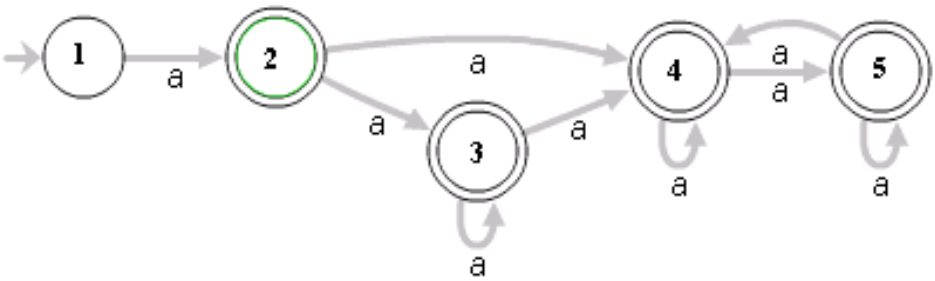

Figure 1: NFA of (a+)+

This attack was only made possible because in *auth.js* the *validatePassword* function does name.match(pass), which checks if the password is included in the username, something that should be done. However, *match()* works with patterns, matching a string to a regular expression. To prevent this from happening, we simply have to use a function that checks that the username does not contain the password but does not use Regex. This function is *includes()* and it performs a case-sensitive search to determine whether a given string may be found within another.

1.2 Task 2: Web Side Channels

1.2.1 Identifying Storage Side Channels

Before diving into the task, we should first review what constitutes a *storage side channel*. A classic example is a registration form that responds differently when you attempt to register an already-taken username, leaking information about existing accounts.

In our target application, *Forumerly*, we suspected multiple storage side channels related to the registration and parking features (according to the task). We began by examining the "passport.js" file that, among other things, handles the registration. The code in that file validates inputs in sequence and returns the very first error it encounters:

- Username check: Error message: "Username is already in use." (see figure 2)
- License-plate check: Error message: "License plate already registered by other user." (see figure 3)
- Password check: Error messages for invalid passwords (checked last)

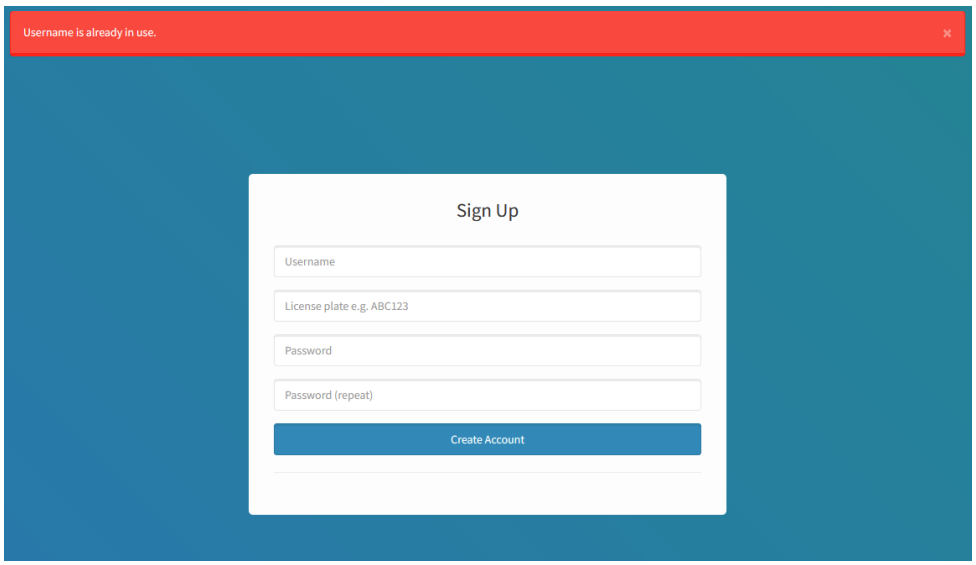

Figure 2: Username Side Channel

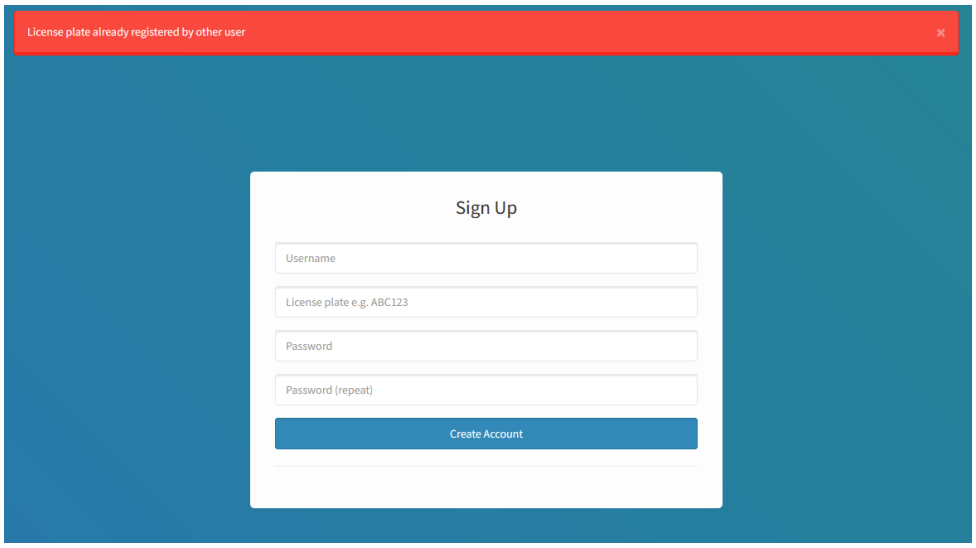

Figure 3: License Plate Side Channel

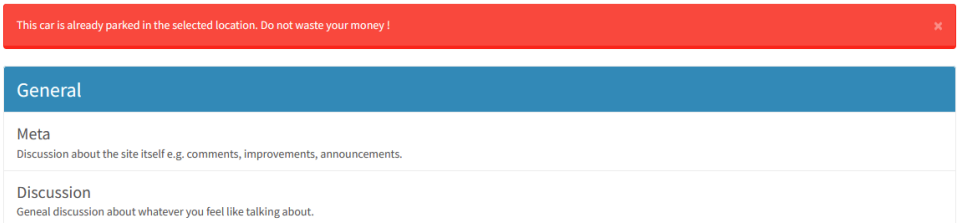

Figure 4: Location Side Channel

Because the handler immediately halts on the first error and displays it on the webpage, an attacker can infer which validation step failed by observing which error appears. Even if errors were hidden, an attacker could measure the response times (fewer correct checks = faster response) and thus infer on which step the registration failed.
Additionally, we would have discovered the side channel simply by using the graphic web interface (with random inputs).

A similar side channel exists in the parking workflow. After logging in, if you attempt to park a car that is already registered at the selected location, the system replies: "This car is already parked in the selected location. Do not waste your money!" (see figure 4)
This distinct message (and its corresponding processing path) again leaks information about where each license plate is parked.

1.2.2 Exploiting the Side Channels

We automated exploitation with a Python script that exploits each side channel.
The usernames side channel we exploit by inputting a wordlist of common usernames (usernames.txt) and then we submit each candidate to the registration endpoint via a post request and record which ones trigger the "Username is already in use" error. For the license plates we generate all valid plate combinations and then submit each candidate until we find plates already registered (limited to the first 50 for speed, but easily adjustable to cover the full space). Lastly, the parking locations we iterate through all possible parking locations for every registered license plate that we found through the previous side channel and attempt to park it (post request). Afterwards, we record the location that triggers "This car is already parked in the selected location".
By combining these steps, the script enumerates active usernames, in-use license plates, and their current parking spots

1.2.3 Mitigation Strategies

To fix these storage side channels, we should implement the following things.
Username and license-plate checks:

- Always return a single, generic error message (e.g., "Registration failed; please check your inputs.") without revealing which field caused the failure. (For usability, you may still document field requirements separately.)
- Perform all validation steps on every request before returning an error, and then insert a random or fixed delay so that timing does not leak which check failed first.
- Ensure the backend always performs the same database operations (real or dummy lookups) even on early-exit cases, preventing cache or I/O timing leaks.

Parking-location checks:

- Only accept parking requests for plates owned by the authenticated user, rejecting any attempts to submit other users' plates. (This changes Forumerly's logic; if that's not allowed, you must always return the same response—whether success or error—and pad the response time so it's constant.)
- Replace the distinct "already parked" message with a generic confirmation (e.g., "Your parking request was processed."). Still apply uniform processing delays and identical code paths: if the car was already parked, you simply skip re-parking internally but never reveal that decision. This way the only thing learned post-request is that "now the car is parked here," but nothing about its prior state.

1.3 Task 3: Cross Site Scripting (XSS)

1.3.1 Identifying the XSS

Before starting with the task we made ourselves familiar with Express.js, mustache.js and the overall function of the Forumerly web server. During this we discovered the thread post shown in 5, where html tags are used to get bold and other writing.

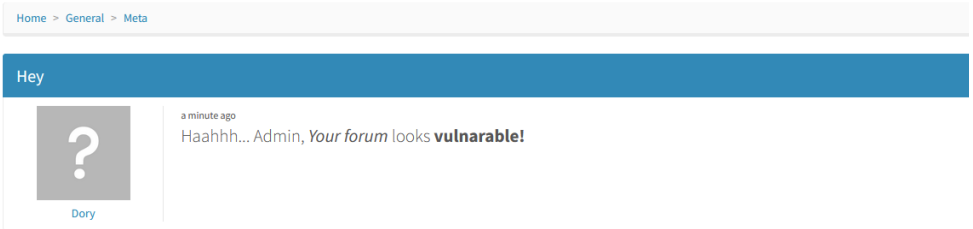

Figure 5: Thread Post

Upon inspecting the code connected with the body of thread posts we noticed that line 37 in the "thread.hjs" file has a triple mustache (only the body of a new thread, not replies), which is vulnerable according to the mustache.js documentation. However, at the same time we also found that the input is sanitized (replaces "<script", "<img", "<svg" and "javascript:" in "forum.js"). This is obviously not enough sanitization, which mean XSS is possible here. After this discovery we started crafting and testing inputs that avoid sanitization, while also exploiting weaknesses in the CSP configuration.

1.3.2 Exploiting the XSS

After discovering that we can download the AngularJS script because "cdnjs.cloudflare.com" is allowed in the CSP. We managed to forge the following script:

```
<Script src=
  "https://cdnjs.cloudflare.com/ajax/libs/prototype/1.7.2/prototype.js"
  ></script>
<sCript
   src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.0.8/angular.js"
   /></script>
<div ng-app ng-csp>
 {{$on.curry.call().alert(1)}}
</div>
```

This is a simple script that simply gives us an alert. However, we could also embed a link to anywhere or immediately redirect the user upon opening the thread post that contains the XSS (not limited by CSP). An example for redirection upon opening the thread post:

```
<Script src=
  "https://cdnjs.cloudflare.com/ajax/libs/prototype/1.7.2/prototype.js"
  ></script>
<sCript
   src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.0.8/angular.js"
   /></script>
<div ng-app ng-csp>
 {{$on.curry.call().location.href='https://owasp.org/www-community/
   attacks/xss/'}}
</div>
```

The Content Security Policy (CSP) can be found in the "app.js" file on lines 48-52. It allows 'self' for "defaulSrc", meaning that by default, nothing is allowed from any external domain. Additionally, it has "scriptSrc: ["'self'", "use.fontawesome.com", "ajax.googleapis.com", "cdnjs.cloudflare.com"]", which overrides the default for script tags. Therefore, in addition to our own origin, we also permit JavaScript files from these sources.
This means that we can bypass the CSP by using scrips from one of these three provided sources, which in our case are the angular.js and prototype.js scripts.

1.3.3 Impact and Mitigation

The XSS we found means that the attacker can redirect users that open the attacker's thread to wherever the attacker wants. However, he cannot actually steal cookies from the website, because the cookies are set to httpOnly and we also cannot send requests to the server because of the CSP.

To address the detected XSS issue, we needed to improve the input sanitization. The previous implementation could be bypassed simply by altering the case of characters (e.g., `script -> sCript`). To properly sanitize user input while still allowing basic formatting (such as bold and italic text), we implemented an allowlist-based sanitization approach. Specifically, we now permit only the `<b>` and `<i>` tags. The updated sanitization logic can be found in `/forumerly/routes/forum.js` at lines 13 to 23 and line 235. We commented out line 235 to demonstrate the exploit, because otherwise the vulnerability would be mitigated and is thus not reproducible.
We also tightened the CSP by removing unused or potentially dangerous external script

sources. Specifically, `cdnjs.cloudflare.com` — which is not used anywhere in Forumerly and is used in the exploit presented in section 1.3.2 — has been removed from the CSP. Additionally, the `ajax.googleapis.com` source is also known to enable XSS attacks (the `use.fontawesome.com` is not known to open such attack vectors) and should thus, also be removed. However, the `head.js` file uses a script from this source. Therefore, we downloaded that specific script, removed google from the CSP and changed the link reference in `head.js` to the local version of the used script. Although `use.fontawesome.com` remains in the policy, we will adopt the same approach (serve a local copy and remove the remote source) if any vulnerability is discovered.

Lastly, I do not believe that one can set up a real web application to prevent the execution of any malicious script. However, one can make the applications as secure as possible and very hard to find an entry point for attackers, e.g. by combining strict allowlist sanitization with a minimal, self-hosted CSP (ideally only allowing `'self'`). This ensures a balance between security and functionality for a real-world web application.

1.4 Task 4: Remote Code Execution (RCE)

1.4.1 Identifying the prototype pollution vulnerability

In *users.js* in the function *merge()*, we can receive a source directly from the client because the route */upload/users* does not have any admin requirements. This means that when a user posts a JSON to this route, it is directly uploaded to the database. If one of the keys of the file is "__proto__" then we can modify the Object.prototype itself, incurring prototype pollution.

1.4.2 Identifying the RCE vulnerability

In *passport.js*, in the *authenticate()* function, as we are parsing the JSON file provided by the user, we use *eval()* on the input, a function that evaluates JavaScript code represented as a string and returns its completion value. This means that anything that is in the *options.userAutoCreateTemplate* is run as if it were JavaScript. The only problem now is how to add code there.

1.4.3 Exploiting the prototype pollution and RCE

In order to start this attack, we first need to register a new user. Then, we POST a JSON file to the */upload/users* route updating that user and modifying *userAutoCreateTemplate*

so that it has some code that we want to run (a Reverse Shell payload, e.g). Then we only need to login with a random username and password (they don't have to exist) and because *options.userAutoCreateTemplate* is not defined on the object itself, it falls back to the polluted prototype.

1.4.4 Mitigation

In order to mitigate this attack we can do three things. Firstly, we can disregard any keys in the JSON related to prototype such as "__proto__", "constructor", and "prototype". Secondly, we can make it so that only the admin is able to upload JSON files directly by adding *adminRequired* in the post parameters. Lastly we could entirely remove the *eval()* from the code, but this would only prevent RCE attack and not the prototype pollution.

2 Contributions

Eva Sophia Schreyer wrote: Side Channel Task, XSS Task and the corresponding reports (including the setup of the report).

Tomás Sobral Teixeira wrote: ReDoS Task and RCE Task and the corresponding reports + proofreading of report.